Sample/Reference ReadMEs:

```
# Density-Aware k-Anonymity for Location Privacy in IoT Smart Cities

## Project Overview

This project implements **Density-Aware k-Anonymity** for ensuring user location privacy in **IoT-enabled smart cities**.
Unlike traditional graph-based k-anonymity, which uses a fixed k, Density-Aware k-Anonymity dynamically adjusts the anonymity parameter based on local user density, providing an optimized **privacy-utility tradeoff**.

The system models an urban environment as a grid-based graph and generalizes user locations into **connected anonymization regions** that contain at least *k* users, where k is chosen adaptively based on the local spatial context.

---

## System Architecture

### Core Components
```

1. SmartCityGraph
Models the smart city as a 2D grid graph:
- Nodes represent intersections; edges represent road connectivity
- Randomly distributes a configurable number of users across nodes
- Maintains spatial coordinates for realistic visualization and distance metrics

2. Density-Aware k-Anonymity Algorithm
The core logic engine for density-aware privacy:
- **Density Computation:** Uses BFS (depth=1) to count users in a node's immediate neighborhood
- **Adaptive k Selector:** Maps neighborhood density to privacy levels (k=10 for sparse, k=5 for medium, k=2 for dense)
- **Region Expansion:** Grows a connected subgraph until the required *k* users are reached

3. Density-Aware k-Anonymity Experiment
An experiment management layer that:
- Orchestrates multiple simulation runs (default: 20 runs)
- Collects statistical data on densities, k-values, and resulting region sizes
- Provides summary metrics including average region size and min/max bounds

4. Density-Aware k-Anonymity Visualization
A dedicated visualization suite that:
- Generates scatter plots for privacy-utility analysis
- Produces spatial graph plots highlighting the target user and the anonymized region
- Automatically exports results to a structured `results/` directory

---

Density-Aware k-Anonymity Algorithm

Algorithm Outline

```text
For each user query:
1. Identify the user's target node in the SmartCityGraph
2. Compute local user density using a 1-hop BFS search
3. Select k value adaptively:
   - Density < 4  -> k = 10 (High Privacy)
   - Density < 10 -> k = 5  (Balanced)
   - Else         -> k = 2  (High Utility)
4. Perform BFS region expansion starting from the target node
```

```
5. Accumulate users until the selected k-threshold is met
6. Return the resulting connected subgraph as the privacy region

```

Key Properties

* Context-aware anonymity levels based on real-time user distribution
* Guaranteed connectivity of anonymization regions via graph-based BFS
* Higher service precision in dense urban areas; higher privacy in sparse regions
* Modular, object-oriented design for research extensibility

Implementation Features

* Object-oriented architecture (City, Algorithm, Experiment, and Visualization classes)
* Automated result export to local `results/` folder
* Configurable simulation parameters (grid size, user count, random seed)

* Multi-run statistical analysis pipeline for Density-Aware k-Anonymity
* High-fidelity visualization of spatial anonymization regions

---

Density-Aware k-Anonymity Experiment Results

Performance Metrics (Example Output)

Run ID	Target Node	Density Class	Selected k	Region Size
Run 01	12	Sparse	10	14
Run 05	04	Medium	5	6
Run 18	22	Dense	2	1

Observations

* **Adaptive Efficiency:** The system successfully lowers k in dense areas to preserve location utility.
* **Privacy Assurance:** In sparse areas, the region size expands significantly to ensure k-anonymity.

* **Connectivity:** All generated regions maintain road-network connectivity, preserving spatial realism.

Visualization Outputs

1. Density vs. Selected k

A scatter plot demonstrating the inverse relationship between local density and the required anonymity level.

Output file: `results/density_vs_k.png`

2. Selected k vs. Region Size

Visualizes how the physical area of the privacy region scales with the chosen k-value in the Density-Aware k-Anonymity framework.

Output file: `results/k_vs_region_size.png`

3. Region Visualization

A spatial graph highlighting:

```
* **Yellow Node:** The actual user location (Target)
* **Red Nodes:** The connected anonymization region
* **Blue Nodes:** The rest of the smart city grid

**Output file:** `results/region_visualization.png`

---

## Quick Start

### Prerequisites

```bash
pip install networkx matplotlib

```

Running the Simulation

```bash
python density_aware_k_anonymity_simulation.py

```

Generated Files

* results/density_vs_k.png
```

* results/k_vs_region_size.png
* results/region_visualization.png

Technical Details

Graph Model

* 5 × 5 Grid-based city model (configurable)
* Integer-labeled nodes with (x, y) mapping
* Random user distribution with seed support for reproducibility

Privacy Computation

* BFS-based density estimation (depth = 1)
* BFS-based region expansion (connected subgraph constraint)
* Threshold-based adaptive k selection

Metrics Collected

* Neighborhood user density
* Region size (node count)
* Summary statistics (Average k, Avg/Min/Max region size)

```
---

## Current Limitations

### Topological Complexity

* Grid-based model is a simplification of real-world irregular road networks
* BFS density is limited to a fixed 1-hop depth

### Dynamic Constraints

* Model currently assumes static users (no mobility/trajectory support)
* Temporal privacy correlations are not modeled

---

## Future Work

### Algorithmic Extensions

* Implementation of l-diversity and t-closeness on top of Density-Aware k-Anonymity
* Adaptive BFS depth for density estimation
* Differential privacy integration for edge weights

### System Integration
```

* Support for real-world OpenStreetMap (OSM) graph data
* Integration with fog/edge computing nodes for distributed anonymization
* Trajectory anonymization for moving IoT devices

Research Contributions

* Modular OOP framework for Density-Aware k-Anonymity research
* Empirical validation of density-aware privacy selection
* Automated visualization and data collection pipeline
* Extensible baseline for IoT location privacy in smart cities

---

References and Context

This work builds on concepts from:

* Spatial cloaking and graph-based anonymization
* Location privacy in IoT smart city architectures

* Density-aware privacy-utility optimization

Author: Praagya Garg
Context: IoT Smart Cities Privacy Research
Date: January 2026

Differential Privacy Location Obfuscation for IoT Smart Cities

Project Overview

This project presents a **differential privacy–based location obfuscation framework** for protecting user and device location privacy in **IoT-enabled smart cities**.
The system applies the **Laplace mechanism** to perturb coordinate-based location data, providing **formal ε-differential privacy guarantees** while preserving statistical utility for smart city analytics.

The objective is to **balance privacy and utility**, enabling aggregate location-based

services and data analysis without revealing exact device positions.

System Architecture

Core Components

1. DifferentialPrivacyLocationObfuscator

Implements ε-differential privacy using the Laplace mechanism:

- Adds calibrated noise to individual location coordinates
- Uses the privacy budget parameter ε to control noise magnitude
- Smaller ε provides stronger privacy at the cost of higher utility loss

2. IoTSmartCitySimulator

Models a realistic smart city IoT environment:

- Simulates 50 IoT devices across a 10 × 10 city area

- Generates realistic spatial clustering around points of interest
- Supports heterogeneous device types (smartphones, sensors, cameras, vehicles)

3. PrivacyUtilityAnalyzer

Evaluates privacy–utility tradeoffs using:

- Location error (Euclidean distance)
- Statistical error metrics (mean, median, standard deviation, maximum)
- Privacy level categorization based on ε values

Differential Privacy Algorithm

Algorithm Outline

```text
For each location (x, y):
1. Compute noise scale = sensitivity / ε
2. Sample Laplace noise for x and y independently
3. Generate obfuscated coordinates:
   x' = x + Laplace(0, scale)
   y' = y + Laplace(0, scale)
4. Return obfuscated location (x', y')
```

```

---

## Key Properties

- Formal ε-differential privacy guarantee
- Privacy does not depend on the number of users
- Resistant to post-processing and background knowledge attacks
- Privacy–utility tradeoff is explicitly tunable via ε

---

## Implementation Features

- Coordinate-level differential privacy using the Laplace mechanism
- Configurable privacy budgets (ε = 0.1, 0.5, 1.0, 2.0, 5.0)
- Realistic IoT device distribution and clustering
- Comprehensive statistical and visual privacy–utility analysis
- Exportable results for reproducibility

---
```

Simulation Results

Performance Metrics by Privacy Level

ε (epsilon)	Privacy Level	Mean Error	Median Error	Max Error	Std Dev
0.1	Maximum Privacy	16.65	12.14	63.48	12.56
0.5	Very High Privacy	3.29	2.92	9.06	2.19
1.0	High Privacy	1.61	1.38	5.37	1.10
2.0	Medium Privacy	0.77	0.68	2.00	0.48
5.0	Low Privacy	0.29	0.25	1.05	0.21

Observations

- Location error decreases logarithmically as ε increases
- Strong privacy (ε = 0.1) introduces significant noise
- ε ≈ 1.0 provides a practical balance between privacy and utility

- High ε values preserve accuracy with limited privacy protection

---

Visualization Outputs

1. Location Obfuscation Demonstration

Shows original and obfuscated locations for multiple ε values, with displacement vectors illustrating noise magnitude.

Output file:

```bash
dp_location_obfuscation_demo.png
```

2. Privacy–Utility Analysis

Four-panel analysis including:

- Mean location error vs ε
- Error distribution boxplots
- Standard deviation vs ε
- Privacy category comparison

Output file:

```bash
dp_privacy_utility_analysis.png
```

3. Technical Analysis

Detailed plots showing:

- Noise magnitude distributions
- Noise clouds for different ε values
- Continuous privacy–utility curves
- Device-type-specific error analysis

Output file:

```bash
dp_technical_analysis.png
```

Quick Start

Prerequisites

```bash
```

```
pip install numpy matplotlib
```

Running the Simulation

```bash
python differential_privacy_simulation.py
```

Generated Files

- `dp_location_obfuscation_demo.png`
- `dp_privacy_utility_analysis.png`
- `dp_technical_analysis.png`
- `dp_simulation_results.json`

Technical Details

Laplace Mechanism

- **Sensitivity:** L1 sensitivity = 1.0
- **Noise Scale:** sensitivity / ε
- **Noise Distribution:** Laplace(0, scale)

```
---

### City Simulation

- **City Area:** 10 × 10 units
- **IoT Devices:** 50
- **Deployment Model:**
  - Clustering around predefined points of interest
  - Random spatial noise for realism

---

## Evaluation Metrics

- Euclidean distance for location error
- Mean error
- Median error
- Standard deviation
- Maximum error
- Privacy categorization based on ε (epsilon)
ranges

---

## Current Limitations

### High Noise for Strong Privacy
```

- Very small ε values introduce large location displacement
- May reduce usefulness for individual location-based services

---

Privacy Budget Management

- Repeated queries consume privacy budget
- No built-in budget tracking across applications

Coordinate-Based Obfuscation

- Ignores geographic constraints such as roads and buildings
- May produce physically implausible locations

Uniform Noise Application

- Same noise distribution for all users
- Does not adapt to local density or contextual factors

```
---

## Future Work

### Algorithmic Extensions

- Gaussian differential privacy
- Local differential privacy variants
- Adaptive and personalized ε selection

---

### Geographic Constraints

- Road-network-aware noise generation
- Building and water-body constraints
- Graph-constrained differential privacy

---

### Adaptive Privacy

- Density-aware noise calibration
- Temporal privacy budget management
- Context-aware privacy control

---
```

Hybrid Approaches

- Combination of differential privacy and k-anonymity
- Semantic location protection
- Integration with cryptographic techniques

Research Contributions

- Practical implementation of location differential privacy
- Realistic IoT smart city simulation
- Comprehensive privacy-utility evaluation framework
- Visual analytics for understanding privacy effects
- Modular and extensible research codebase

References and Context

This work builds on foundational concepts in:

- Differential privacy
- Laplace mechanism

```
- Location privacy in IoT systems
- Privacy–utility tradeoff analysis

This framework serves as a **baseline reference**
for studying and comparing probabilistic location
privacy mechanisms in smart city environments.

---

**Author:** Pushkal Gupta
**Context:** IoT Smart Cities Privacy Research
**Date:** January 2026
```

Sample Codes(structure and format)

```python
import networkx as nx
import matplotlib.pyplot as plt
import random
import os
from statistics import mean
from typing import Dict, List, Set, Tuple


#
============================================================
==================
```

```
# Smart City Graph
#
================================================================
==================

class SmartCityGraph:
    """
    Represents a smart city using a 2D grid graph.
    Provides node coordinates for realistic
plotting.
    """

    def __init__(self, grid_size: int = 5,
num_users: int = 30, seed: int = None):
        if seed is not None:
            random.seed(seed)

        self.grid_size = grid_size
        self.graph =
nx.convert_node_labels_to_integers(nx.grid_2d_graph
(grid_size, grid_size))
        self.num_users = num_users

        # Assign simple 2D coordinates for drawing
        self.positions = {node: (node % grid_size,
node // grid_size) for node in self.graph.nodes()}

        # Populate users
```

```
        self.user_at_node = {node: 0 for node in
self.graph.nodes()}
        self._populate_users()

    def _populate_users(self):
        nodes = list(self.graph.nodes())
        for _ in range(self.num_users):
            n = random.choice(nodes)
            self.user_at_node[n] += 1

    def neighbors(self, node: int):
        return list(self.graph.neighbors(node))


#
================================================
==================
# Density-Aware k-Anonymity Algorithm
#
================================================
==================

class DensityAwareKAnonymityAlgorithm:
    """Implementation of the Density-Aware k-
Anonymity logic."""

    def __init__(self, city: SmartCityGraph):
        self.city = city
```

```
    # 1. Density Computation ---------------------
--------------------------
    def compute_local_density(self, node: int,
depth: int = 1) -> int:
        visited = {node}
        queue = [(node, 0)]
        total_users = self.city.user_at_node[node]

        while queue:
            current, d = queue.pop(0)
            if d == depth:
                continue

            for neigh in
self.city.neighbors(current):
                if neigh not in visited:
                    visited.add(neigh)
                    queue.append((neigh, d + 1))
                    total_users +=
self.city.user_at_node[neigh]

        return total_users

    # Density Interpretation ---------------------
--------------------------
    def classify_density_level(self, density: int)
-> str:
        if density < 4:
```

```
            return "Sparse"
        elif density < 10:
            return "Medium"
        return "Dense"

    # 2. Adaptive k selection ---------------------
----------------------------
    def select_adaptive_k(self, density: int) ->
int:
        if density < 4:
            return 10
        elif density < 10:
            return 5
        return 2

    # 3. Region expansion -------------------------
----------------------------
    def expand_anonymization_region(self,
start_node: int, k: int) -> Set[int]:
        region = {start_node}
        queue = [start_node]
        user_count =
self.city.user_at_node[start_node]

        while user_count < k and queue:
            current = queue.pop(0)
```

```
            for neigh in
self.city.neighbors(current):
                if neigh not in region:
                    region.add(neigh)
                    queue.append(neigh)
                    user_count +=
self.city.user_at_node[neigh]
                    if user_count >= k:
                        break

        return region


#
==========================================================
==================
# Density-Aware k-Anonymity Experiment Manager
#
==========================================================
==================

class DensityAwareKAnonymityExperiment:
    """Runs the Density-Aware k-Anonymity
simulation multiple times."""

    def __init__(self, algorithm:
DensityAwareKAnonymityAlgorithm, runs: int = 20):
        self.algorithm = algorithm
        self.city = algorithm.city
```

```
        self.runs = runs

        self.densities: List[int] = []
        self.k_values: List[int] = []
        self.region_sizes: List[int] = []

    def run_simulation(self):
        print("\n====== Running Density-Aware k-
Anonymity Experiment ======\n")
        for i in range(self.runs):
            target =
random.choice(list(self.city.graph.nodes()))

            d =
self.algorithm.compute_local_density(target)
            k = self.algorithm.select_adaptive_k(d)
            region =
self.algorithm.expand_anonymization_region(target,
k)

            self.densities.append(d)
            self.k_values.append(k)
            self.region_sizes.append(len(region))

            print(
                f"Run {i+1:02d} | Target={target} |
Density={d} "
```

```
f"({self.algorithm.classify_density_level(d)}) |
k={k} | Region Size={len(region)}"
            )

        print("\n====== Experiment Complete
======\n")
        return self.densities, self.k_values,
self.region_sizes

    def get_experiment_summary(self):
        return {
            "avg_density": mean(self.densities),
            "avg_k": mean(self.k_values),
            "avg_region_size":
mean(self.region_sizes),
            "max_region_size":
max(self.region_sizes),
            "min_region_size":
min(self.region_sizes)
        }


#
==========================================================
==================
# Density-Aware k-Anonymity Visualization
```

```
#
============================================================
==================

class DensityAwareKAnonymityViz:
    """Visualization suite for the Density-Aware k-
Anonymity results."""

    @staticmethod
    def ensure_results_folder():
        if not os.path.exists("results"):
            os.makedirs("results")

    @staticmethod
    def plot_density_vs_k(density, k):

DensityAwareKAnonymityViz.ensure_results_folder()
        plt.figure()
        plt.scatter(density, k, color="darkblue")
        plt.xlabel("Local Density")
        plt.ylabel("Selected k")
        plt.title("Density-Aware k-Anonymity:
Density vs k")
        plt.grid(True)
        plt.savefig("results/density_vs_k.png",
dpi=300)
        plt.show()
```

```
    @staticmethod
    def plot_k_vs_region_size(k, region_size):

DensityAwareKAnonymityViz.ensure_results_folder()
        plt.figure()
        plt.scatter(k, region_size, color="green")
        plt.xlabel("Selected k")
        plt.ylabel("Anonymization Region Size
(nodes)")
        plt.title("Density-Aware k-Anonymity: k vs
Region Size")
        plt.grid(True)
        plt.savefig("results/k_vs_region_size.png",
dpi=300)
        plt.show()

    @staticmethod
    def visualize_specific_region(city:
SmartCityGraph, region: Set[int], target: int,
density: int, k: int):

DensityAwareKAnonymityViz.ensure_results_folder()

        pos = city.positions
        plt.figure(figsize=(7, 7))
        nx.draw(city.graph, pos, with_labels=True,
node_color="lightblue")
```

```
        # region nodes (excluding target)
        region_others = [node for node in region if
node != target]
        nx.draw_networkx_nodes(city.graph, pos,
nodelist=region_others, node_color="red")

        # target node highlighted
        nx.draw_networkx_nodes(city.graph, pos,
nodelist=[target], node_color="yellow")

        plt.title(
            f"Density-Aware k-Anonymity
Region\nTarget={target}, Density={density}, k={k},
Region Size={len(region)}"
        )

plt.savefig("results/region_visualization.png",
dpi=300)
        plt.show()


#
==========================================================
==================
# MAIN EXECUTION
#
==========================================================
==================
```

```
def main():
    # 1. Initialize the City and the Algorithm
    smart_city = SmartCityGraph(grid_size=5,
num_users=30, seed=0)
    algorithm =
DensityAwareKAnonymityAlgorithm(smart_city)

    # 2. Run the Experiment
    experiment_manager =
DensityAwareKAnonymityExperiment(algorithm,
runs=20)
    density_data, k_data, size_data =
experiment_manager.run_simulation()

    # 3. Print Summary Statistics
    print("=== Density-Aware k-Anonymity Summary
===")
    for key, value in
experiment_manager.get_experiment_summary().items()
:
        print(f"{key}: {value:.2f}")

    # 4. Generate Analytics Plots

DensityAwareKAnonymityViz.plot_density_vs_k(density
_data, k_data)
```

```
DensityAwareKAnonymityViz.plot_k_vs_region_size(k_d
ata, size_data)

    # 5. Visualize a Sample Run
    sample_node =
random.choice(list(smart_city.graph.nodes()))
    sample_density =
algorithm.compute_local_density(sample_node)
    sample_k =
algorithm.select_adaptive_k(sample_density)
    sample_region =
algorithm.expand_anonymization_region(sample_node,
sample_k)


DensityAwareKAnonymityViz.visualize_specific_region
(
        smart_city, sample_region, sample_node,
sample_density, sample_k
    )


if __name__ == "__main__":
    main()



#!/usr/bin/env python3
```

```
"""
Differential Privacy Location Obfuscation for IoT Smart Cities
================================================================
This implementation demonstrates location privacy using differential privacy
with Laplace noise addition to coordinate-based location data.

Author: Pushkal Gupta
Date: January 2026
"""

import numpy as np
import matplotlib.pyplot as plt
from matplotlib.patches import Circle
import random
import json
import math
from typing import List, Tuple, Dict


# ----------------------------------------------------------------
# Differential Privacy Obfuscator
# ----------------------------------------------------------------
```

```python
class DifferentialPrivacyLocationObfuscator:
    """Implements differential privacy for location
data using Laplace mechanism."""

    def __init__(self, epsilon_values: List[float]
= [0.1, 0.5, 1.0, 2.0, 5.0]):
        """
        Initialize the differential privacy
obfuscator.

        Args:
            epsilon_values: List of privacy budget
values (smaller = more private)
        """
        self.epsilon_values = epsilon_values
        self.sensitivity = 1.0  # L1 sensitivity
for location coordinates

    def add_laplace_noise(self, value: float,
epsilon: float) -> float:
        """
        Add Laplace noise to a single coordinate
value.
        """
        scale = self.sensitivity / epsilon
        noise = np.random.laplace(0, scale)
        return value + noise
```

```
    def obfuscate_location(
        self, x: float, y: float, epsilon: float
    ) -> Tuple[float, float]:
        """
        Add differential privacy noise to location
coordinates.
        """
        noisy_x = self.add_laplace_noise(x,
epsilon)
        noisy_y = self.add_laplace_noise(y,
epsilon)
        return noisy_x, noisy_y

    def batch_obfuscate(
        self, locations: List[Tuple[float, float]],
epsilon: float
    ) -> List[Tuple[float, float]]:
        """
        Obfuscate multiple locations with the same
privacy parameter.
        """
        return [
            self.obfuscate_location(x, y, epsilon)
            for x, y in locations
        ]


# ----------------------------------------------
--------------------
```

```
# Smart City Simulator
# ------------------------------------------------
------------------

class IoTSmartCitySimulator:
    """Simulates IoT devices in a smart city for
location privacy testing."""

    def __init__(self, city_size: float = 10.0,
num_devices: int = 50):
        self.city_size = city_size
        self.num_devices = num_devices
        self.device_locations =
self._generate_device_locations()
        self.device_types =
self._assign_device_types()

    def _generate_device_locations(self) ->
List[Tuple[float, float]]:
        locations = []

        poi_centers = [
            (2.0, 2.0),
            (8.0, 3.0),
            (5.0, 7.0),
            (3.0, 8.0),
            (7.0, 8.0),
        ]
```

```
        clustered_devices = int(0.7 *
self.num_devices)
        random_devices = self.num_devices -
clustered_devices

        for _ in range(clustered_devices):
            center = random.choice(poi_centers)
            x = np.random.normal(center[0], 0.8)
            y = np.random.normal(center[1], 0.8)
            x = max(0, min(self.city_size, x))
            y = max(0, min(self.city_size, y))
            locations.append((x, y))

        for _ in range(random_devices):
            x = random.uniform(0, self.city_size)
            y = random.uniform(0, self.city_size)
            locations.append((x, y))

        return locations

    def _assign_device_types(self) -> List[str]:
        device_types = [
            "smartphone",
            "vehicle_gps",
            "smart_camera",
            "sensor_node",
            "wearable",
```

```
        ]
        weights = [0.4, 0.2, 0.15, 0.15, 0.1]
        return np.random.choice(
            device_types, size=self.num_devices,
p=weights
        ).tolist()


# ------------------------------------------------
------------------
# Privacy-Utility Analyzer
# ------------------------------------------------
------------------

class PrivacyUtilityAnalyzer:
    """Analyzes privacy-utility tradeoffs."""

    @staticmethod
    def calculate_location_error(
        original: Tuple[float, float],
        obfuscated: Tuple[float, float],
    ) -> float:
        return math.dist(original, obfuscated)

    @staticmethod
    def calculate_privacy_level(epsilon: float) ->
str:
        if epsilon >= 5.0:
            return "Low Privacy"
```

```
        elif epsilon >= 2.0:
            return "Medium Privacy"
        elif epsilon >= 1.0:
            return "High Privacy"
        elif epsilon >= 0.5:
            return "Very High Privacy"
        else:
            return "Maximum Privacy"

    @staticmethod
    def calculate_utility_loss(errors: List[float])
-> Dict[str, float]:
        return {
            "mean_error": np.mean(errors),
            "median_error": np.median(errors),
            "std_error": np.std(errors),
            "max_error": np.max(errors),
            "min_error": np.min(errors),
        }


# ------------------------------------------------
------------------
# Simulation
# ------------------------------------------------
------------------

def run_differential_privacy_simulation():
```

```
    print("Differential Privacy Location
Obfuscation for IoT Smart Cities")
    print("=" * 70)

    city_sim =
IoTSmartCitySimulator(city_size=10.0,
num_devices=50)
    dp_obfuscator =
DifferentialPrivacyLocationObfuscator()
    analyzer = PrivacyUtilityAnalyzer()

    print("Smart City Simulation Setup:")
    print(f" City Size: {city_sim.city_size} x
{city_sim.city_size} units")
    print(f" IoT Devices: {city_sim.num_devices}")
    print(f" Device Types:
{set(city_sim.device_types)}")
    print(f" Privacy Levels (ε):
{dp_obfuscator.epsilon_values}")
    print("-" * 70)

    results = {
        "epsilon_values":
dp_obfuscator.epsilon_values,
        "privacy_levels": [],
        "mean_errors": [],
        "median_errors": [],
        "std_errors": [],
```

```
        "utility_metrics": {},
    }

    for epsilon in dp_obfuscator.epsilon_values:
        print(f"\nTesting Differential Privacy with
ε = {epsilon}")

        obfuscated_locations =
dp_obfuscator.batch_obfuscate(
            city_sim.device_locations, epsilon
        )

        errors = [
            analyzer.calculate_location_error(orig,
obf)
            for orig, obf in zip(
                city_sim.device_locations,
obfuscated_locations
            )
        ]

        utility_metrics =
analyzer.calculate_utility_loss(errors)
        privacy_level =
analyzer.calculate_privacy_level(epsilon)


results["privacy_levels"].append(privacy_level)
```

```
results["mean_errors"].append(utility_metrics["mean
_error"])

results["median_errors"].append(utility_metrics["me
dian_error"])

results["std_errors"].append(utility_metrics["std_e
rror"])
        results[f"epsilon_{epsilon}"] =
utility_metrics

        print(f" Privacy Level: {privacy_level}")
        print(f" Mean Location Error:
{utility_metrics['mean_error']:.3f}")
        print(f" Median Error:
{utility_metrics['median_error']:.3f}")
        print(f" Std Deviation:
{utility_metrics['std_error']:.3f}")
        print(f" Max Error:
{utility_metrics['max_error']:.3f}")

    return city_sim, dp_obfuscator, results


# ----------------------------------------------------
--------------------
# Visualizations
```

```python
# ----------------------------------------------------------------------

def create_privacy_visualizations(
    city_sim: IoTSmartCitySimulator,
    dp_obfuscator: DifferentialPrivacyLocationObfuscator,
    results: Dict,
):
    fig1, axes = plt.subplots(2, 3, figsize=(18, 12))
    fig1.suptitle("Differential Privacy Location Obfuscation Comparison")

    vis_epsilons = dp_obfuscator.epsilon_values

    for idx, epsilon in enumerate(vis_epsilons):
        ax = axes[idx // 3, idx % 3]

        obfuscated_locs = dp_obfuscator.batch_obfuscate(
            city_sim.device_locations, epsilon
        )

        orig_x, orig_y = zip(*city_sim.device_locations)
        obf_x, obf_y = zip(*obfuscated_locs)
```

```
        ax.scatter(orig_x, orig_y, c="blue",
alpha=0.7, label="Original")
        ax.scatter(obf_x, obf_y, c="red",
alpha=0.7, label="Obfuscated")

        for i in range(min(10, len(orig_x))):
            ax.plot(
                [orig_x[i], obf_x[i]],
                [orig_y[i], obf_y[i]],
                color="gray",
                alpha=0.5,
            )

        privacy_level =
PrivacyUtilityAnalyzer.calculate_privacy_level(epsi
lon)
        mean_error = results["mean_errors"][idx]

        ax.set_title(
            f"ε = {epsilon} ({privacy_level})\nMean
Error: {mean_error:.2f}"
        )
        ax.set_xlim(-1, city_sim.city_size + 1)
        ax.set_ylim(-1, city_sim.city_size + 1)
        ax.grid(True)
        ax.legend()

    plt.tight_layout()
```

```
    plt.savefig("dp_location_obfuscation_demo.png",
dpi=300)

    return fig1


# ------------------------------------------------
--------------------
# Main
# ------------------------------------------------
--------------------

def main():
    city_sim, dp_obfuscator, results =
run_differential_privacy_simulation()

    create_privacy_visualizations(city_sim,
dp_obfuscator, results)

    with open("dp_simulation_results.json", "w") as
f:
        json.dump(results, f, indent=2)

    print("\nSimulation completed successfully.")
    plt.show()


if __name__ == "__main__":
    main()
```

[redacted]